Development, Simulation, and Performance Evaluation of a Distributed Leader Election Protocol in Java

COMP212 | February 23, 2026

Zhibin Jiang

Since the asynchronous LCR algorithm (without subnets) and the ring-within-ring asynchronous LCR algorithm represent a step-by-step progression, the latter (ring-within-ring asynchronous LCR) **includes the asynchronous LCR algorithm**, provided we do not consider subnets (or set the number of interfaces to 0). Therefore, the following sections will introduce how to generalize from the asynchronous LCR algorithm to an asynchronous LCR algorithm with subnets!

1 Terminology Description

The overall network structure can be divided into two types of processors: interface processors and non-interface processors. Below, we refer to the sub-rings of the "ring-within-ring" as subnets. The number of interface processors is equal to the number of subnets; each subnet consists of a certain number of **non-interface processors**, and an interface processor manages its own subnet ring structure.

Additionally, we call the round in which a processor sends its first message the **wakeup round**, or we say the processor wakes up in this round.

Processors are equivalent to nodes; the term "processor" will be used hereafter.

2 Wakeup Logic

In the standard LCR algorithm, the default start round is $round = 0$, but in the asynchronous LCR algorithm, the starting round may be uncertain! First, for the asynchronous LCR algorithm, the difficulty lies in the fact that an unawakened `processor` **cannot** receive or respond to messages!

If the standard LCR algorithm is used (e.g., each processor only broadcasts during its initial round or when it receives an `id` larger than its own `myId`), it will lead to data loss, causing the entire ring to fall into an infinite loop.

> Consider an example with 2 processors. Suppose processor P_1 has a larger `myId` but starts earlier, while P_2 has a smaller `myId` but starts later. When P_1 wakes up and sends its own `myId`, P_2 does not receive it. When P_2 wakes up and sends its `myId` to P_1, P_1 will ignore it. Thereafter, they fall into a deadlock-like situation: P_1 is waiting for P_2 to forward a larger `myId`, while P_1 never received a larger `myId` due to **message loss**. Meanwhile, P_2 is also waiting for P_1 to send a larger `myId`. Because the `myId` P_2 received from P_1 was smaller, P_2 simply ignored it, resulting in no response.

To solve this problem, we stipulate that every processor must confirm the other party has awakened before sending a message; otherwise, it will not send.

Therefore, we only need to add several flags to each processor:

Variable Name	Type	Description
`nextWakenUp`	`boolean`	Whether the next `processor` has awakened
`prevWakeUp`	`boolean`	Whether the previous `processor` has awakened

Of course, we stipulate that when each `processor` wakes up, it should send an "I have awakened" signal to both its **next** `processor` and **prev** `processor`. In this case, if we wish to send our `myId` to the next processor, we must first determine if it has awakened. If it has not, we should not send any data to it (to prevent loss) until we receive its wakeup signal.

The corresponding message types are:

Variable Name	Description
`TELL_NEXT_ACTIVATION_SIGN_SLOT`	Inform next processor current is awake.
`TELL_PREV_ACTIVATION_SIGN_SLOT`	Inform previous processor current is awake.

> See `Slot.java` in the `Constant.java` package.

However, some issues still persist. For example, consider two processors P_1 and P_2, where the former wakes up early and the latter late. When P_1 wakes up, P_2 is not yet active, so P_2 cannot receive P_1's wakeup message. When P_2 eventually wakes up, P_1 receives P_2's wakeup message and thus confirms P_2 is active. But at this point, P_2 does not know P_1 is awake! Because P_2 is unsure whether its message to P_1 was delivered, this leads to an information asymmetry.

Therefore, we need to add a message type for **acknowledging the other party's awakening**! This way, once sends "I acknowledge receipt of your (processor) awakening" can naturally conclude that is awake.

Variable Name	Description
`ACK_NEXT_ACTIVATION_SIGN_SLOT`	Ack to `next` received
`ACK_PREV_ACTIVATION_SIGN_SLOT`	Ack to `prev` received

> Note: When we send our own wakeup signal to the next processor, we use the `NEXT` field; therefore, for the recipient to acknowledge it back to us (their previous processor), they must use the `ACK_NEXT` field.

Through this, we ensure that for an asynchronously started LCR, all processors can set each other to "awakened" after a certain number of rounds. This conclusion can be proven via mathematical induction. Suppose processor P has a previous and a next processor (which might be the same processor in the boundary case where the network only has two nodes); the following three scenarios exist:

- P **is the first to wake up:**
  - **Scenario 1:** P's next processor wakes up. P receives the wakeup request, marks the next processor as awakened, and responds. Then P's previous processor wakes up, and the same logic applies.

  - **Scenario 2:** P's previous processor wakes up first; the steps are the same as above.

  Therefore, if P wakes up first, P will inevitably succeed in setting both neighbors to "awakened."
- **P wakes up after the previous processor but before the next (or vice-versa):**
  Since P wakes up before the next processor, it will eventually receive the next processor's wakeup request and set its status correctly. Since P wakes up after the previous processor, it misses the initial wakeup message. However, as soon as P wakes up and sends its own wakeup message to the previous processor, the previous processor responds with an `ACK`, allowing P to set the previous processor to "awakened."
- **P is the last to wake up:**
  In this case, as soon as P wakes up and sends requests to both neighbors, they will return `ACK` signals, allowing P to know they are both awake.

Consequently, in any situation, P will eventually successfully identify the awakening status of its neighbors while minimizing network transmissions. By induction, this can be generalized to the entire ring network; thus, the algorithm is correct.

3 Information Transmission

In this algorithm, while awakening signals are transmitted bidirectionally, all other information is unidirectional. This design choice simplifies the code. The `Processor` class has only one input buffer, `Message[] receivedSlots`. Bidirectional communication would require more complex `Message` design to avoid overwriting and increase logic coupling. By using unidirectional logic, we ensure that the received `leaderId` is the result of a single-direction traversal.

Every processor sends awakening notifications to both sides upon waking up. However, a processor only passes the `leaderId` to the **next** processor. Since synchronous LCR is already proven correct, the asynchronous version simply adds asynchronous waiting: each processor ensures the next is awake before sending, preventing message loss.

4 Subnets

To upgrade the asynchronous LCR to include subnets, we apply the asynchronous LCR within the subnet to ensure correctness. Before the subnet elects a leader, the interface processor remains in an **unawakened** state (by setting `startRound` to infinity). Once the subnet election is complete, the `startRound` is set to the current round to activate the main ring election, and its `myId` is set to the maximum `id` elected by the subnet to represent the subnet in the main ring.

Additional logic: When an interface processor receives a `myId`, it checks if it matches the subnet's maximum `id`. If it does, the interface processor becomes the `leader`, representing the node from its subnet that won the global election.

5 Termination

When the election concludes, the leader sends a **Termination** signal, which is a specific message type.

Only the leader initially sends this signal (then terminates itself). Subsequently, all other processors terminate and forward the signal immediately upon receipt. We can guarantee the following:

Variable Name	Description
`TERMINATION_SLOT`	Termination signal

1. **Leader Identity Consistency:** Once a processor receives the termination signal, its current `leaderId` is guaranteed to be the actual global `leaderId` (provable by induction).
2. **Liveness (Termination):** After a finite number of rounds, all processors in the network will terminate.

Proof of Liveness: Since the signal is issued by the leader, it implies its `myId` has already traversed the entire main ring. At this stage, no unawakened processors can exist; therefore, the `<TERMINATION>` signal does not need to check if the next node has terminated.

Proof of Correctness: Since the leader's `myId` (the maximum ID) has completed a full cycle and the algorithm logic only retains the maximum ID, the correctness is consistent with the standard LCR algorithm proof.

6 Summary

The Algorithm in `processor.run()`: Based on the proof above, the following logic is executed by each processor in every round:

If the processor is a Non-Interface Processor:

- If the awakening round hasn't been reached, **return**.
- Otherwise, perform **First-round forwarding**.

If the processor is an Interface Processor:

- If the subnet election is not finished, simulate one round of the subnet election.
- **Subnet status check:**
  - **Finished:** Set `myId` to the subnet's `winnerId`, represent the subnet in the main ring, and set `startRound` to current.
  - **In-progress:** Set `startRound` to the next round (or ∞).
- If `startRound` is current, perform **First-round forwarding**; else, **return**.

First-round Forwarding Logic:

- **Notification:** Notify neighbors of awakening (write to send buffers).
- **Propagation:** Send `myId` to the next processor.
- **Buffer Processing:**
  Termination: Forward signal and terminate.
  LCR Logic:
  i. $id = self$: **Leader elected**. Send `TERMINATION` and stop.
  ii. $id < currentLeader$: Ignore.
  iii. $id > currentLeader$: Update `leaderId` and forward.
  Awakening: Respond with corresponding `ACK` to neighbors.

- **Finalize:** Dispatch buffers to the respective neighbors.

7 Code Architecture

7.1 Simulator Class

The simulator is implemented in the `Simulator.java` class within the `NetworkAndSimulator` package. Each `Simulator` object maintains the following state:

Variable Name	Type	Description
`rings`	`List<Processor>`	Bidirectional ring.
`hasNextRound`	`boolean`	Has next round?

The simulator holds the **entire ring** (either the main ring or a subnet). It iterates through the `List<>` of `processor` objects, invoking their respective `run()` methods. The actual logic is encapsulated within the `processor` itself; importantly, an individual `processor` has no visibility into the entire ring.

> **Note: The simulator serves only to synchronize the rounds; it does not participate in the actual communication or decision-making logic.**

Non-interface processors are treated as standard processors in an asynchronous LCR environment. Interface processors are specialized via the `processor.setAsInterfaceProcessor()` method, which enforces the following:

- `startRound` is initialized to a maximum value and is only set to the current round upon subnet completion.
- No initial `leaderId` is assigned (initialized to minimum).
- The processor type is explicitly set to "interface processor."

The `Simulator` class provides the `.generateMainRing()` method to construct the network using the parameters detailed below:

Variable Name	Type	Description
`mainRingSize`	`int`	Size of the main ri
`startBound`	`int`	Random starts rou bound.
`interfaceProcessorNumber`	`int`	Number of interf processors.
`subnetSize`	`List<Integer>`	Size of each subnet.
`shuffledId`	`int[]`	Shuffled unique ids.

ID distribution is managed by `Shuffle.java` from the `Tool` package. It populates an array from 1 to N and performs an $O(N)$ shuffle. This shared ID array ensures global uniqueness across all processors. Furthermore, each interface processor generates an additional `Simulator.java` instance as a subnet attribute.

Variable Name	Type	Description
`myId`	`int`	Unique ID
`startRound`	`int`	Awakening round.
`leaderId`	`int`	Leader id.
`status`	`ProcessorStatus`	Status: `UNKNOWN`, `LEADER`, or `LOST`.
`subnetSimulator`	`Simulator`	subnet.

7.2 Processor Class

The core method of `Processor.java` is `.run()`, implementing the asynchronous LCR with subnets. Key member variables:

A `Processor` can only access the next/previous `Processor` via `.getMyId()`, simulating a realistic distributed environment.

7.3 Network Class

The `Network` class manages the communication layer to reduce overall code complexity. Processors are restricted to sending and receiving requests exclusively to and from their direct neighbors. This constraint is implemented using a `HashMap`, where the key is a processor's `id` and the value is the message directed to it.

To simulate asynchronous network delays, the `Network` employs two distinct buffer variables: `thisRound` and `nextRound`. A message dispatched during the current round is strictly placed in the `nextRound` buffer, ensuring it remains unavailable to the recipient until the subsequent round begins.

When a processor pulls a specific message, such as `<348, M1>`, the network immediately deletes it from the buffer. Upon the invocation of the `nextRound()` method, the following state transition occurs to prevent the caching of stale messages:

1. The `thisRound` buffer is completely cleared.
2. Data from the `nextRound` buffer is moved into `thisRound`.
3. The `nextRound` buffer is subsequently cleared to prepare for incoming messages.

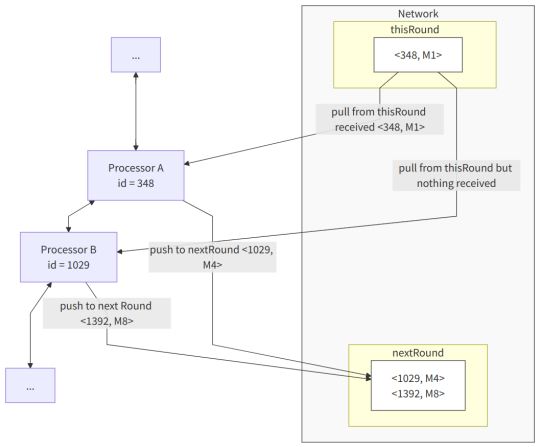

Figure 1: Network Class Architecture

8 Experimental Setup

There are many parameters available for study, such as the size of the main ring, the startup time of each processor, the number of interface processors, the size of each subnet, and so on. In this section, we do not discuss the `ID` distribution, which is implemented using a random distribution. Instead, we investigate the

network structure (e.g., with or without subnets) and network scale (number of processors).

8.1 Increasing the Number of Processors

Network scale: The simulation ranges from 1 to 1000 processors. However, please note that due to computational constraints, `MULTIPLIER=1` was used in the code implementation (`Experiment.java` class). The following assumes the use of `MULTIPLIER = 10`, meaning the scale is increased tenfold (from 1 to 10000)!

8.1.1 Task A: Structure Without Subnets

We explore the impact of the number of processors in the main ring as a variable on algorithm complexity in a state without subnets (i.e., no interfaces).

This is equivalent to the impact of Delayed Start LCR (without subnets) on algorithm complexity within a fixed random startup time. The parameters are as follows:

Parameter	Value
`mainRingSize`	$[1, 10000]$
`startBound`	200
`interfaceProcessorNumber`	0

8.1.2 Task B: Structure With Subnets

We explore the impact of the number of main ring processors as a variable on algorithm complexity in the presence of subnets, with a fixed percentage of interfaces.

Assume the number of subnets (or interfaces) is 10% of the main ring size, and specify that each subnet size is a value within $[3, 20]$.

Parameter	Value
`mainRingSize`	$[1, 10000]$
`startBound`	200
`interfaceProcessorNumber`	$0.1 \times$ mainRingSize
`List<Integer> subnetSize`	$[3, 20]$

8.2 Fixed Total Number of Processors

While the total number of processors (including interface processors, non-interface processors, and subnets) is fixed, we can still adjust certain parameters for research.

8.2.1 Task C: Increasing the Number of Subnet Processors

Fix the number of interface processors but increase the number of processors within the subnets.

Fix the total processor size at 10000. Assuming there are 1000 interface processors and each subnet has at least 3 processors, the total number of subnet processors ranges from $[3000, 9000]$. Consequently, the number of non-interface processors varies inversely from 6000 to 0.

The variable here is the total number of subnet processors, denoted as x. Thus, the main ring size should be $10000 - x$.

To achieve an even distribution for `List<Integer> subnetSize`, the `generateRandomAssigned` method in the `Shuffle` class is utilized.

Parameter	Value
`mainRingSize`	$10000 - x$
`startBound`	200
`interfaceProcessorNumber`	1000
`List<Integer> subnetSize`	Total of x

8.2.2 Task D: Increasing the Number of Interface Processors

Fix the number of subnet processors but increase the number of interfaces.

Fix the total processor size at 10000. Assuming the number of subnet processors is fixed at 6000, the number of interface processors will be in the range $[1, 2000]$. Let the number of interface processors be x; then the main ring size is $10000 - 6000 - x = 4000 - x$.

Parameter	Value
`mainRingSize`	$4000 - x$
`startBound`	200
`interfaceProcessorNumber`	x
`List<Integer> subnetSize`	Fixed total 6000

9 Experimental Evaluation and Findings

9.1 Proof of Experimental Correctness

The `Simulator` class implements a `validation()` method. If the network state does not satisfy the condition—"all processors are defeated except for the leader, and all processors' `leaderId` matches the maximum `leaderId`"—a runtime exception is thrown to halt execution. This function is invoked in two critical stages to guarantee algorithm correctness:

1. It is called immediately after an interface processor's internal subnet completes its local election.

2. It is called in `Experiment.java` after each iteration and before the network reset to ensure the validity of the (x, y_1, y_2) data points.

9.2 Performance Evaluation

This evaluation assesses the correctness and performance of the algorithm across various network configurations, employing `numpy` for regression analysis. During each execution, the simulator records the number of rounds (Rounds) and the total number of messages transmitted (Transmission) until termination. To determine the asymptotic time and communication complexity, we plotted the experimental data and performed curve fitting against standard functions like n, $n \log n$, and n^2.

9.2.1 Task A Analysis: No Subnet Structure

In this task, we simulated the LCR algorithm with delayed starts in a single ring (no interface processors), with the Main Ring size x increasing from 1 to 1000.

- **Rounds**: Based on the fitting results, the relationship is $y = -1.36 \times 10^{-5}x^2 + 2.0158x + 182.77$. The negligible quadratic coefficient confirms a **linear $O(n)$ growth**, matching the physical limit of message propagation in a ring.

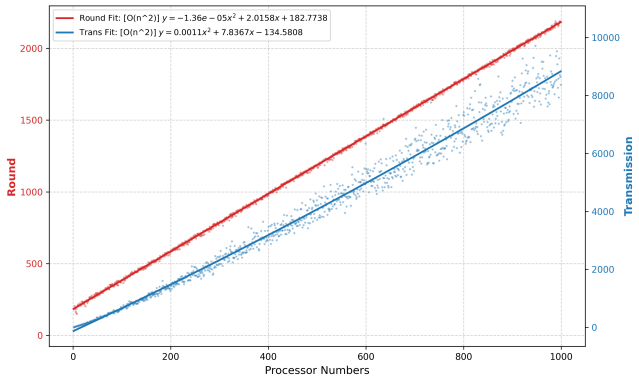

Figure 2: Task A: Experimental data (top) and regression fitting (bottom).

- **Transmission**: The message count fits $y = 0.0011x^2 + 7.8367x - 134.58$. Under random IDs and asynchronous starts ([0, 200]), the communication complexity follows a **quadratic** $O(n^2)$ **growth**, consistent with standard LCR theory.

9.2.2 Task B Analysis: With Subnet Structure

This task introduces a "ring-in-ring" structure. Interface processors are fixed at 10% of the main ring size, with subnets randomly sized between [3, 20].

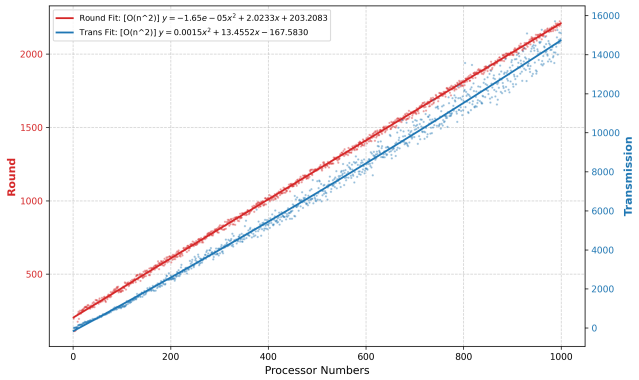

Figure 3: Task B: Performance with subnets (top) and regression fitting (bottom).

- **Complexity Trend**: The trends mirror Task A, with rounds at $O(n)$ and messages at $O(n^2)$ ($y = 0.0015x^2 + 13.4552x - 167.58$).
- **Performance Overhead**: The addition of numerous non-interface processors increases the baseline message count, though the small quadratic coefficient keeps the actual growth relatively manageable.

9.2.3 Task C Analysis: Increasing Subnet Processors

With the total processors (10,000) and interface processors (1,000) fixed, we increased the number of processors within subnets (x-axis: 300–900).

- **Optimization**: As subnets grow, the main ring shrinks. Both rounds and transmissions show a **significant downward trend**.
- **Reasoning**: Subnets execute elections in parallel. Shrinking the main ring mitigates the $O(n^2)$ penalty of the global election, effectively offloading communication costs to parallel clusters.

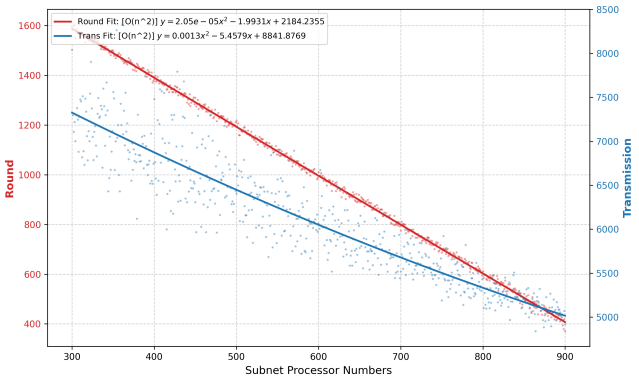

Figure 4: Task C: Effect of subnet size on total performance.

9.2.4 Task D Analysis: Increasing Interface Processors

With the total processors (10,000) and subnet processors (6,000) fixed, we increased the number of interface processors (x-axis: 0–200).

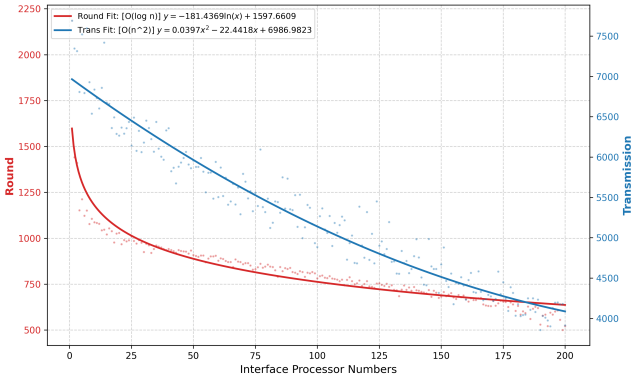

Figure 5: Task D: Performance benefits of increasing interface nodes.

- **Logarithmic Rounds**: Rounds exhibit a near-perfect $O(\log n)$ **decline** ($y = -181.4369\ln(x) + 1597.66$).
- **Communication Efficiency**: Transmissions plummet quadratically as the network is partitioned into more, smaller parallel rings, accelerating synchronization via shared variables.

9.3 Performance Summary

1. **Baseline**: Without optimization, asynchronous LCR exhibits $O(n)$ time and $O(n^2)$ communication complexity.
2. **Topology Impact**: A "Main Ring + Subnets" structure significantly optimizes performance by converting serial bottlenecks into parallel computations.

10 Conclusion

This simulation successfully implemented a distributed ring-in-ring leader election algorithm in Java. We verified that regardless of topology or ID distribution, the system correctly identifies the global maximum ID. The results demonstrate that increasing interface processors and optimizing subnet scale can effectively reduce message redundancy and time latency, providing a valuable reference for distributed protocol design.